

MODULE 4 - HANDOFF

Parallel Lane Orchestration - Runbook

Hard rules, escalation paths, and when to kill a lane.

Version: v1 - 2026-05-09

Author: Chase Whited / Whited Consulting

Stable URL: raw.githubusercontent.com/cwhited26/aipocketagency-brain

Parallel Lane Orchestration - Runbook

Multi-agent work moves fastest when independent lanes run in parallel and dependent work stays serial. This runbook is the working operator playbook for splitting work into lanes, firing them, and rejoining cleanly without losing changes.

1. What a lane is

A lane is a coherent batch of 3-10 items that share a UI surface, a migration, or a logical dependency. Each item ships as one or more logical commits. Lanes can run in parallel only when they touch disjoint files or fully isolated database tables.

If two lanes both edit the same component file or the same migration, they are not parallel - they are serial work that needs an explicit ordering.

2. Hard rules

- **One agent per file.** Two agents touching the same file at once produces conflicts that are expensive to resolve. Plan around this.
- **One migration in flight at a time.** Database schema is shared state. Lane order for migrations is strict and announced.
- **No auto-fire without greenlight.** Lane plans land as locked docs with status "awaiting greenlight on fire order." Nothing fires until the operator says fire.
- **Each lane logs to the brain on completion.** Daily Log + Change Log + Decision Log (if applicable) + Feature Inventory.
- **Verification claims must specify the layer.** Lane-report / disk / remote / behavior-verified. "Browser smoke test NOT performed" is the canonical phrasing for incomplete verification.

3. Lane plan format

Every lane plan is a single markdown doc in the brain. Required sections:

- Lane number and short title.
- Items in the lane (numbered, one line each).
- Files this lane touches (so collisions with other lanes are visible).
- Dependencies (which lanes must land first, if any).
- Recommended fire order (a recommendation, not a trigger).
- Verification expectations (what tests, which layer, which UI flows).
- Rollback path (what to revert if the lane has to back out).

4. Firing a lane

Operator says fire. The orchestrator (Claude or human) does the following in order:

- Confirm the lane plan is the current locked version.
- Confirm dependent lanes have landed (check Change Log SHAs).
- Hand the plan to the executing agent (Codex or Claude Code) with the standard handoff bridge.

- Watch for the executing agent's confirmation that brain context is loaded.
- Standby - do not start a new conflicting lane until the executing agent reports completion.

5. When to kill a lane

Kill a lane immediately if any of these happen:

- The executing agent is making changes outside the lane's declared file list.
- A migration in the lane fails on a staging database and the agent attempts a destructive recovery (DROP, DELETE, manual data fix).
- Build is broken on main for more than ten minutes after the lane committed.
- Two consecutive verification failures with no diagnosis.
- The executing agent stops referencing brain context (signs of context-window drift).

Kill protocol: stop the agent, capture state in a session handoff doc, decide whether to revert or fix forward, log the kill in Daily Log.

6. Escalation paths

Symptom	First step	If not fixed in 10 minutes
Build broken on main	Identify the breaking commit; revert if isolated.	Stop all lanes touching the broken area; surface to operator.
Migration failed on staging	Roll forward only if additive. If destructive, freeze.	Operator reviews; do not run destructive recovery without sign-off.
Two agents racing on the same file	Pause the second agent; resolve who owns the file for this lane.	Move one lane's items to a follow-up lane.
Brain write conflict (ETag mismatch)	Re-fetch, re-apply on top of latest.	If the conflict keeps recurring, an upstream agent is over-writing - pause and reconcile.
Sentry / error monitor flooded	Triage by error key; identify the lane that introduced it.	Roll the relevant commit back; fire a fix lane.

7. Rejoining lanes

Lanes split, run in parallel, and rejoin at a known checkpoint. The rejoin is explicit - an integration commit that confirms all lanes' changes are present in main and tests pass with the merged state. Skipping the rejoin produces silent drift that takes weeks to surface.

Standard rejoin protocol: each lane appends a one-line entry to the integration doc when complete. When all expected entries are present, fire the integration commit (run full test suite, tag the SHA, log to Daily Log, update Feature Inventory).

8. The runbook in one paragraph

Plan lanes as locked docs. Fire only on operator greenlight. One agent per file, one migration at a time. Log every lane to the four files on completion. Kill any lane that drifts; capture state, decide revert or fix-forward. Rejoin explicitly. Never trust a lane self-report - verify at the layer the claim implies.